

MLC-Norm: Optimizing LayerNorm and RMSNorm on Apple M4

Ketsia Kemkuini

Faculty of Mathematics and Computer Science
Friedrich-Schiller-Universität Jena
Jena, Germany

gertrude.kemkuini.fouodji@uni-jena.de

Mariza Sitcheu

Faculty of Mathematics and Computer Science
Friedrich-Schiller-Universität Jena
Jena, Germany

mariza.yamdjeu.sitcheu@uni-jena.de

Abstract

Normalization is a standard component of Transformer architectures, and operations such as LayerNorm and RMSNorm combine low arithmetic intensity with substantial data movement, making them suitable targets for low-level optimization. This work develops hand-written and JIT-generated implementations for the Apple M4 using Arm SME and Streaming SVE and integrates them into a tiled-execution IR runtime. Every reported implementation is checked against an FP64 reference before timing. The measurements show that the limiting factors change with working-set size. At 256 MiB, processing four SVL row blocks together raises RMSNorm throughput from 10.04 to 21.05 GiB/s and LayerNorm from 7.38 to 13.09 GiB/s. SME2 multi-vector accesses further increase RMSNorm throughput to 24.63 GiB/s at 256 MiB, but provide almost no benefit at 16 MiB. Variants using the ZA array reduce nominal memory traffic, yet remain slower than the grouped SSVE design after an SME2 redesign. The JIT-generated kernels reproduce their hand-written counterparts and match their throughput at the two larger benchmark shapes. Separately, the final auto-dispatched implementation reaches $1.87\text{--}2.12\times$ the strongest measured PyTorch or ExecuTorch LayerNorm median throughput and $5.38\text{--}5.97\times$ for RMSNorm at one thread.

Keywords: SME, Streaming SVE, SME2, Apple M4, LayerNorm, RMSNorm, JIT code generation, memory bandwidth.

1 Introduction

Efficient execution of modern machine-learning workloads depends not only on arithmetic throughput, but also on how data movement, reductions, and vector execution are mapped to the target architecture. Transformer architectures provide a particularly relevant setting because they are built from repeated tensor operations whose performance is shaped by both computation and memory behavior [3]. Among these operations, LayerNorm [1] and RMSNorm [2] combine relatively low arithmetic intensity with reduction-heavy computation. They therefore provide a compact but technically rich case study for low-level optimization on architectures such as Arm SME.

This work studies LayerNorm and RMSNorm on the Apple M4 through a sequence of hand-written and JIT-generated implementations. Rather than focusing on a single optimized kernel, we examine the effect of different design choices across working-set sizes, whether approaches that reduce data movement actually translate into higher throughput, and how closely runtime-generated code can match the performance of equivalent hand-written implementations. The goal is therefore not only to improve performance, but also to identify which optimizations remain effective across different execution regimes.

The experiments expose a regime-dependent optimization path. Independent accumulation chains are most useful in the cache-assisted case, while grouping four adjacent row blocks becomes the dominant change at the larger working set. SME2 multi-vector accesses are then beneficial mainly for large RMSNorm. A separate ZA design tests whether retaining activations can

reduce re-reads, and the JIT path tests whether the same low-level organization can be produced at runtime. The final kernels are also evaluated against PyTorch and ExecuTorch and under TEIR-controlled parallel scheduling.

The remainder of this paper is organized as follows. Section 2 introduces the normalization operators, the relevant SME features and related work. Section 3 presents the data layout, kernel variants and JIT/TEIR integration. Section 4 describes the experimental platform, correctness gates and performance methodology. Section 5 evaluates the kernel variants, numerical behavior, code generation and framework comparisons. Section 6 brings the measurements together into a regime-dependent interpretation and discusses scheduling and validity limits. Section 7 concludes the paper.

2 Background and Related Work

2.1 LayerNorm and RMSNorm

For a row $x \in \mathbb{R}^N$, LayerNorm computes

$$y_i = \gamma_i \frac{x_i - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \beta_i \quad (1)$$

with

$$\mu = \frac{1}{N} \sum_i x_i, \quad \sigma^2 = \frac{1}{N} \sum_i (x_i - \mu)^2. \quad (2)$$

γ and β are learned affine parameters and ε is a small constant used for numerical stability.

RMSNorm removes mean centering and instead computes

$$y_i = \gamma_i \frac{x_i}{\sqrt{\frac{1}{N} \sum_i x_i^2 + \varepsilon}}. \quad (3)$$

The difference between the two formulations directly affects their implementation. In the LayerNorm formulation used in this work, the mean is computed first, followed by a second pass to accumulate the centered variance and a third pass to produce the output. RMSNorm requires only a sum-of-squares reduction followed by the output pass. The activation input is therefore traversed three times for LayerNorm and twice for RMSNorm.

These additional passes are only part of the performance difference. Both operators contain reductions, whose accumulations introduce dependencies between successive iterations. LayerNorm also contains two dependent reduction stages because the centered variance cannot be evaluated until the mean is known. The kernel design must therefore consider both data access and the execution of the reductions.

The choice of variance formulation also affects numerical behavior. Computing the variance as $E[x^2] - \mu^2$ can lose accuracy when the two terms are close. The LayerNorm implementation instead evaluates the variance from centered values in a second pass. Welford’s recurrence is evaluated separately as an alternative formulation in Section 5.3.

2.2 SME and Streaming SVE

Arm’s Scalable Matrix Extension (SME) introduces a streaming execution mode, scalable vector operations, and the ZA storage array. SME2 extends this model with additional instructions, including multi-vector memory operations [7]. In this paper, SVE instructions executed in streaming mode are referred to as Streaming SVE (SSVE).

Three architectural features are particularly relevant to the implementations studied here:

- **Streaming vector length.** The vector width is determined at runtime rather than fixed in the program. The measured SVL is 512 bits, corresponding to 16 FP32 elements. Kernel block sizes are derived from this value instead of assuming a fixed width.
- **ZA storage.** SME provides the ZA array as additional storage while streaming mode is active. We investigate whether retaining activation values in ZA can avoid additional memory accesses between stages of the normalization kernels.
- **SME2 multi-vector operations.** SME2 can access several adjacent scalable vectors with a single instruction. This allows the memory-instruction structure to change without changing the amount of data processed.

Our normalizations use these features primarily for vectorized reductions and elementwise computation rather than matrix operations. Their concrete use in the different kernel variants is described in Section 3.

2.3 Related Work

LayerNorm was introduced by Ba, Kiros, and Hinton [1], while RMSNorm was later proposed by Zhang and Sennrich as a simpler normalization method without mean centering [2]. Their work primarily studies the role of these operators in neural-network models, whereas this paper focuses on their low-level CPU implementation.

Optimized normalization implementations are also available through machine-learning frameworks and inference runtimes. PyTorch provides native and compiled CPU execution paths, while ExecuTorch targets on-device inference and can delegate operators to optimized backends such as XNNPACK [8, 9]. Our work complements those higher-level execution paths by examining LayerNorm and RMSNorm at the level of the generated machine code.

Following the Roofline principle of comparing a kernel with the resource ceilings relevant to its execution [4], the evaluation uses a measured SSVE bandwidth reference for the reduction and elementwise loops studied here. A STREAM-style scale-add probe [6] is measured across working-set sizes, and kernel throughput is compared with the SSVE value at the corresponding size. Matrix-compute peak throughput is not used because these normalization kernels do not execute matrix products.

For numerical stability, Welford’s recurrence provides an alternative to variance formulations based on $E[x^2] - \mu^2$ [5]. It is included in the evaluation as a comparison with the centered two-pass LayerNorm formulation.

3 Kernel Design and Code Generation

3.1 Data layout

Let the activation tensor contain M rows and N features. The implementation uses a column-major layout with an explicit leading dimension,

```
a[row + col * ld].
```

Values from adjacent rows of the same feature are therefore contiguous in memory, whereas consecutive features of one logical row are separated by ld elements. LayerNorm and RMSNorm are mathematically defined across the feature dimension, so directly vectorizing along a logical row would require strided memory accesses. The chosen layout instead makes adjacent rows the natural vectorization dimension.

Figure 1 shows the resulting mapping. Logically, the normalization statistics for one row are obtained by traversing its N features. Physically, the kernel fixes one feature column and loads values from several adjacent rows with a contiguous vector access. It then advances to the next feature while preserving the association between vector lanes and rows. Each vector lane therefore accumulates the normalization statistics of one row throughout the reduction.

This mapping has several consequences for the kernel design:

- **Contiguous vector accesses.** Each feature step loads values from adjacent rows instead of gathering strided values from one logical row.
- **Independent row accumulations.** Each vector lane accumulates statistics for one row, allowing several row reductions to progress simultaneously.
- **No horizontal reduction across lanes.** When the feature traversal finishes, each lane already contains the statistic for its corresponding row, so the row statistics do not need to be combined across the vector.

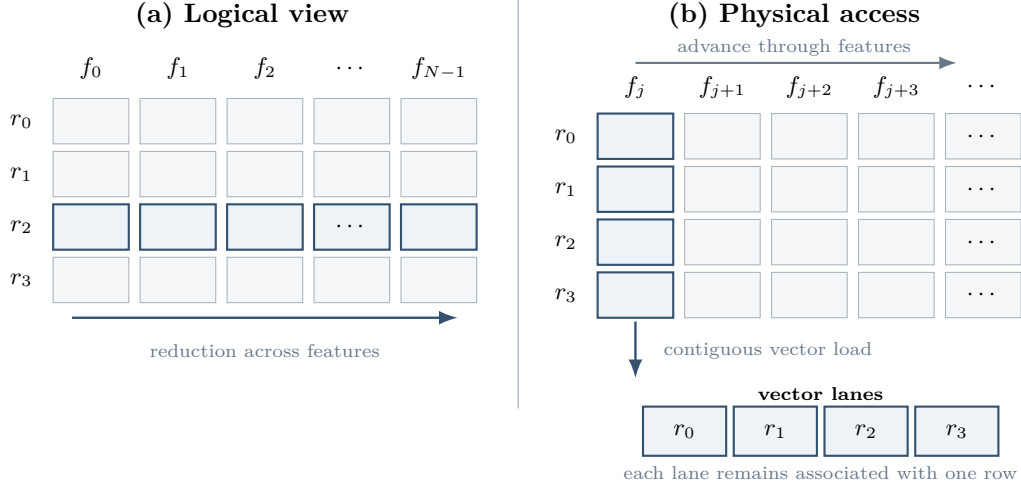


Figure 1. Logical and physical views of the activation layout. On the left, normalization is defined across the features of one row. On the right, the column-major layout makes values from adjacent rows of one feature contiguous in memory. The kernel therefore vectorizes across rows and preserves the lane-to-row mapping while advancing through the feature dimension.

The same lane-to-row mapping is reused when producing the normalized output. Predication masks rows outside the valid range in the final block, allowing incomplete blocks to remain on the vectorized path without a separate scalar tail. This data layout and vector mapping form the common basis for the SSVE kernel variants introduced in the following subsection.

3.2 Optimization path

Starting from the row-vectorized layout described in Section 3.1, the kernel is refined through a sequence of variants. The changes are incremental: early variants modify arithmetic or instruction scheduling, while the later variants change how several row blocks are processed and how their memory accesses are expressed. This makes it possible to associate performance changes with specific design choices.

The variants can be grouped into three stages:

- **Arithmetic and reduction scheduling (V0–V5).** These variants retain the same basic row-block organization while changing reciprocal-square-root evaluation, scalar work, reduction accumulators, and load scheduling.
- **Row-block grouping (V6).** Four consecutive SVL row blocks are processed together, increasing the amount of contiguous data accessed for each feature column.
- **SME2 multi-vector access (V7).** The four-block organization of V6 is retained, but adjacent vectors are loaded and stored using SME2 multi-vector instructions.

Table 1 summarizes the individual variants. The labels are used throughout the evaluation.

Among the early variants, V4 is the most structurally important. A single reduction accumulator creates a dependency from one update to the next. V4 instead maintains four independent partial accumulators and combines them after the feature traversal. This gives the processor several independent arithmetic chains that can be scheduled in parallel. V5 retains this organization

and additionally moves loads earlier in the loop to test whether explicit software pipelining provides further benefit.

The bigger change occurs in V6. Earlier variants process one SVL row block at a time, whereas V6 processes four consecutive blocks together while traversing the same feature dimension. With an SVL of 16 FP32 elements, one row block corresponds to 64 bytes, so each feature step grows from one 64-byte region to four adjacent regions, or 256 bytes in total. Figure 2 illustrates this change.

V7 retains the same four-block organization, addresses, arithmetic, and lane-to-row mapping as V6, but replaces groups of single-vector memory instructions with SME2 multi-vector forms. This makes the V6–V7 comparison a useful controlled experiment: the amount of activation data is unchanged, while the memory-instruction structure is different. Listing 1 shows the corresponding RMSNorm reduction loads.

Listing 1. Memory loads in the RMSNorm V6 and V7 reduction loops.

```
// V6: four adjacent single-vector loads
ld1w {z0.s}, p0/z, [x8]
ld1w {z24.s}, p0/z, [x8, #1, mul v1]
ld1w {z25.s}, p0/z, [x8, #2, mul v1]
ld1w {z26.s}, p0/z, [x8, #3, mul v1]

// V7: same four vectors with one SME2 load
ld1w {z0.s-z3.s}, pn8/z, [x8]
```

The V6 sequence issues four adjacent single-vector loads, while V7 retrieves the same four vectors with one SME2 instruction. A two-vector control, V7x2, is also used to test an intermediate access width. This helps separate the effect of moving from four single-vector accesses to two multi-vector accesses from the effect of collapsing all four vectors into one instruction.

3.3 ZA-based design

The ZA variants explore a different optimization from the previous SSVE progression. While V6 and V7 reorganize accesses to activation memory, the ZA design

Table 1. Kernel variants used in the optimization study. Each variant changes one main aspect of the implementation while retaining the same normalization formulation.

Variant	Main change	Purpose
V0	Baseline SSVE	Process one SVL row block with predicated streaming-vector loops.
V1	FRSQRTE + Newton step	Replace the square-root/division sequence used for the inverse normalization factor.
V2	Hoisted $1/N$	Compute the reciprocal of the feature count once instead of repeating the scalar division inside row-block processing.
V3	Two-column unroll	Test whether ordinary loop unrolling improves overlap in the RMSNorm reduction.
V4	Four independent accumulators	Reduce the dependency between successive accumulation instructions by maintaining several partial sums.
V5	Software-pipelined loads	Test whether explicitly separating loads from their arithmetic use improves instruction scheduling.
V6	Four-block grouping	Process four adjacent SVL row blocks together so that each feature step touches a larger contiguous region of memory.
V7	SME2 multi-vector accesses	Keep the V6 grouping while replacing groups of single-vector memory instructions with SME2 multi-vector forms.

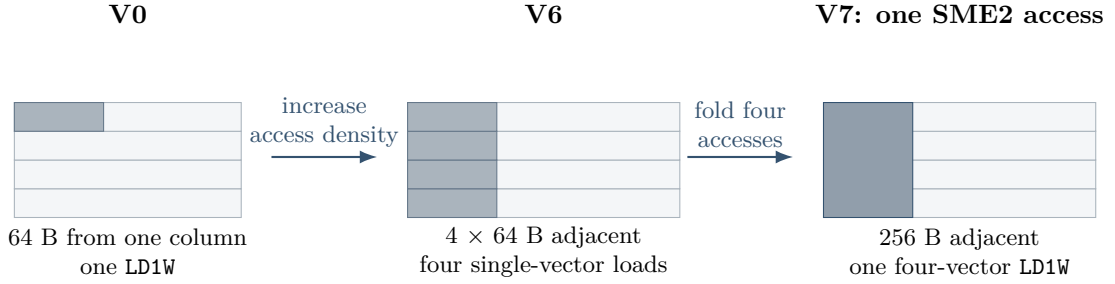


Figure 2. Access pattern behind the main optimization. V6 changes the spatial grouping of the column walk. V7 keeps the same 256 bytes and arithmetic but uses SME2 multi-vector memory instructions.

attempts to keep activation values inside SME state so that they can be reused by a later stage of the normalization computation.

RMSNorm provides the simplest case. Its reduction first consumes the input to compute the sum of squares, after which the same activation values are needed again to produce the normalized output. The conventional implementation loads those values from memory a second time. The ZA variant instead transfers them into ZA while processing the reduction and retrieves them from ZA during output generation. LayerNorm applies the same idea across its additional reduction and output stages.

This does not remove data movement entirely. Reusing values from ZA requires transfers between vector registers and the ZA array, so the usefulness of the approach depends on the cost of those transfers. The organization also differs from the grouped SSVE kernels: the ZA implementation processes one SVL row block at a time, whereas V6 and V7 operate on four adjacent row blocks together. The comparison therefore reflects both the effect of retaining activation values and the different row-block organization.

The initial ZA implementation uses single-vector **MOVA** operations for transfers between vector registers and ZA. A second version replaces these with SME2 multi-vector transfers, allowing the effect of the transfer mechanism to be distinguished from the broader idea of ZA residency. Both implementations are evaluated in

Section 5.2.

3.4 JIT generation and TEIR integration

The optimized normalization kernels are also produced at runtime through a small AArch64 encoding interface. The JIT operates below TEIR: TEIR represents the normalization operation and its iteration space, while the JIT provides the architecture-specific implementation of the leaf computation. This separates tensor-level scheduling from instruction-level code generation.

The emitters construct the machine-code sequence instruction by instruction. Listing 2 shows part of the RMSNorm V6 reduction emitter. The calls correspond directly to the four adjacent SSVE loads and independent accumulation chains of the hand-written kernel.

Listing 2. Excerpt from the RMSNorm V6 JIT emitter.

```
ops.push_back(I::sve_ld1w_imm(I::z0, I::p0, I::x8, 0));
ops.push_back(I::sve_ld1w_imm(I::z24, I::p0, I::x8, 1));
ops.push_back(I::sve_ld1w_imm(I::z25, I::p0, I::x8, 2));
ops.push_back(I::sve_ld1w_imm(I::z26, I::p0, I::x8, 3));
ops.push_back(I::sve_fmla_s(I::z2, I::p0, I::z0, I::z0));
ops.push_back(I::sve_fmla_s(I::z16, I::p0, I::z24, I::z24));
ops.push_back(I::sve_fmla_s(I::z17, I::p0, I::z25, I::z25));
ops.push_back(I::sve_fmla_s(I::z18, I::p0, I::z26, I::z26));
```

The remaining reduction, normalization, control-flow, and prologue/epilogue instructions are emitted in the same manner. The SME2 emitter follows the V7 organization and replaces groups of individual vector memory operations with the corresponding multi-vector encodings. Once emission is complete, the instruction buffer is placed in executable memory and exposed through

the same kernel interface as the hand-written implementation.

At each invocation, the runtime supplies M , N , the input and output leading dimensions, and ε , which determine the loop bounds, address progression, and predicates inside the kernel. These values are derived from the axes and strides declared in the TEIR description, while the surrounding row iteration follows the schedule encoded there. Different problem sizes and outer schedules can therefore reuse the same generated V6 or V7 kernel.

This separation is also what makes the TEIR layer reusable beyond the two normalization operators studied here. The surrounding representation and scheduling machinery does not need to be redesigned for each low-level kernel. A new operation can use the same infrastructure once its semantics and an appropriate leaf lowering or code generator are provided. Architecture-specific instruction selection therefore remains local to the leaf implementation, while tensor shape, iteration structure, and scheduling remain at the TEIR level.

The hand-written kernels are used as references for validating the JIT emitters rather than as shape-specific implementations required by TEIR. The generated instruction words are compared with the machine words of the corresponding linked hand-written kernel. This checks that the emitter produced the intended instruction sequence independently of the numerical correctness tests. Figure 3 summarizes the relationship between TEIR, JIT generation, and this validation path.

The code-generation cost and the performance of the generated kernels are reported in the evaluation. These measurements determine whether the additional generation layer introduces meaningful overhead and whether the emitted kernels retain the performance of their hand-written counterparts.

4 Experimental Methodology

This section describes the experimental platform, benchmark shapes, correctness and timing procedure, throughput metric, bandwidth reference, and external baseline configuration used in the evaluation.

4.1 Platform and benchmark shapes

Measurements were collected on an Apple M4 system running macOS 15.2. The benchmark was compiled with AppleClang 16.0.0 using the Release configuration. The processor reports both `FEAT_SME` and `FEAT_SME2`, and its streaming vector length is 512 bits, corresponding to 16 FP32 elements. The benchmark sets `QOS_CLASS_USER_INTERACTIVE` on the main measurement thread.

The main ablation uses two tensor shapes that fall into different regions of the measured memory hierarchy. For $M = 1024$, $N = 2048$, the combined activation input and output occupy 16 MiB. Increasing the shape to $M = 4096$, $N = 8192$ raises this working set to 256 MiB. A smaller 128×64 configuration is used for experiments concerned with fixed call overhead and small tensors.

The working set in Table 3 is the combined activation input and output,

Table 2. Experimental platform.

Component	Configuration
Processor	Apple M4, 4 P-cores + 6 E-cores
Operating system	macOS 15.2, arm64
Compiler	AppleClang 16.0.0
Build configuration	Release, C++17
SME	<code>FEAT_SME=1</code>
SME2	<code>FEAT_SME2=1</code>
Streaming vector length	512 bits, 16 FP32 lanes
Measurement QoS	<code>USER_INTERACTIVE</code>

Table 3. Principal benchmark shapes and activation working sets.

M	N	working set	use
128	64	64 KiB	small-call experiments
1024	2048	16 MiB	cache-assisted regime
4096	8192	256 MiB	DRAM regime

$$B_{ws} = 2MN \text{ sizeof}(\text{FP32}). \quad (4)$$

The scale and bias vectors are small relative to the activation tensors and are not included in this quantity.

4.2 Correctness and timing protocol

Each configuration is checked before it is timed. The selected kernel is executed on the benchmark shape and its complete output is compared with the FP64 reference. The internal correctness suite uses a relative tolerance of 10^{-5} for the standard kernels and 2×10^{-5} for V4/V5, whose partial-sum reassociation changes FP32 rounding. Variants using the approximate reciprocal square root also allow a 5×10^{-5} absolute margin for outputs close to zero. Timing proceeds only after the corresponding correctness check passes.

For the main kernel measurements, one untimed invocation warms the kernel and its data, followed by 50 timed invocations. Execution time is measured with a high-resolution C++ clock. The minimum execution time is used for the internal kernel ablation. Separate distribution measurements record the median and the 10th and 90th percentiles. Median throughput is used for the external framework comparison.

The JIT-generated kernels additionally undergo the instruction-word comparison described in Section 3.4. Numerical correctness is checked with the same FP64 reference used for the static variants.

4.3 Throughput and bandwidth reference

Kernel throughput is reported in useful GiB/s. For an $M \times N$ FP32 activation, useful bytes are defined as one input tensor and one output tensor,

$$B_{\text{useful}} = 2MN \text{ sizeof}(\text{FP32}) = 8MN \text{ bytes}, \quad (5)$$

giving

$$T_{\text{useful}} = \frac{B_{\text{useful}}}{t}. \quad (6)$$

This quantity provides the same throughput definition for every kernel variant. It should be distinguished

TEIR execution

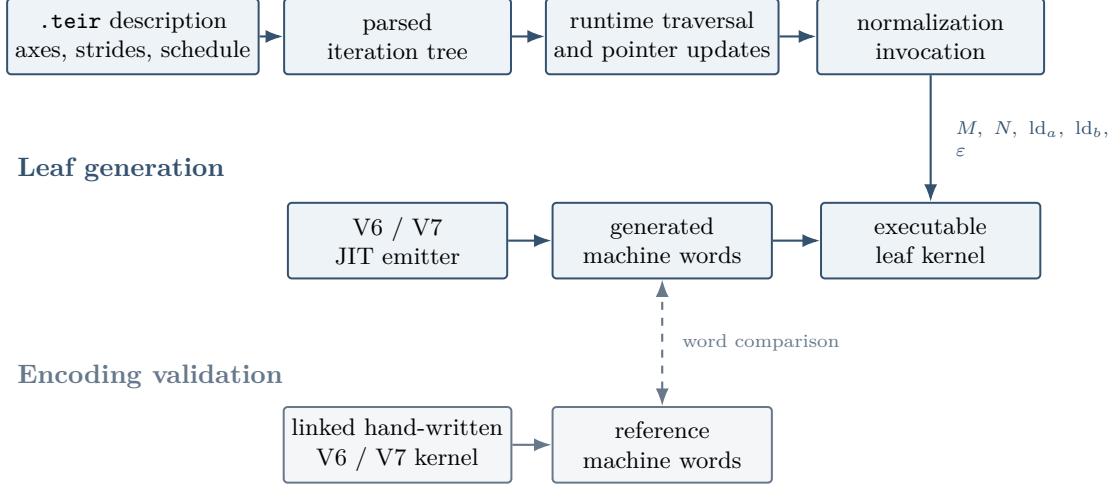


Figure 3. TEIR execution, JIT leaf generation, and encoding validation. The TEIR description specifies axes, strides, and scheduling information, which are parsed into an iteration tree and traversed by the runtime. A normalization invocation passes the derived M , N , leading dimensions, and ϵ to the generated V6 or V7 leaf kernel. The linked hand-written implementation is outside the execution path and is used only to validate the generated machine words.

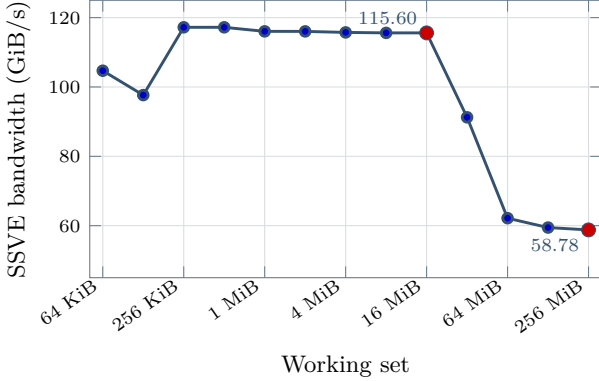


Figure 4. Measured single-core SSVE scale-add bandwidth across working-set sizes. The 16 MiB and 256 MiB points provide the bandwidth references for the two principal ablation shapes.

from the activation traffic generated inside the normalization algorithm. RMSNorm reads the activation during its reduction and again during output generation, giving $2R+1W$. LayerNorm performs the mean reduction, centered-variance reduction, and output stage, giving $3R+1W$. Their activation traffic is therefore $1.5\times$ and $2\times$ the useful-byte quantity, respectively.

The bandwidth reference is measured with an SSVE scale-add probe. Each element is loaded once, incremented, and stored once using contiguous streaming $LD1W/ST1W$ accesses inside a single streaming region. The SSVE probe is timed over 10 repetitions.

The benchmark shapes span substantially different working-set sizes, so the probe is measured from 64 KiB to 256 MiB. Figure 4 shows the resulting curve. At the two working sets used for the main ablation, the measured SSVE bandwidth is 115.60 GiB/s at 16 MiB and 58.78 GiB/s at 256 MiB.

Kernel throughput is compared with the SSVE bandwidth measured at the corresponding working-set size.

The scale-add probe is a $1R+1W$ streaming loop; RMSNorm and LayerNorm perform the additional activation traversals described above. The comparison therefore indicates how kernel throughput relates to the SSVE data rate available in the same working-set regime.

4.4 External baselines and threading controls

PyTorch and ExecuTorch provide the external baselines. PyTorch eager, `torch.compile`, ExecuTorch portable, and ExecuTorch XNNPACK paths are measured as separate configurations and checked against the same numerical reference before timing.

Thread counts are configured explicitly for the threaded comparison. The ExecuTorch driver resets its pthread pool when changing the requested worker count and reads back the resulting configuration before measurement. Process CPU time relative to wall time is also recorded as an indication of concurrent worker activity.

Each framework is measured using its normal operator interface and native tensor layout. MLC-Norm uses the column-major organization described in Section 3.1. The comparison therefore reports the measured execution time of the available LayerNorm and RMSNorm paths for the same operator shapes.

5 Evaluation

The evaluation follows the optimization path from the baseline SSVE kernels to the final SME2 implementation, then examines ZA residency, numerical behavior, JIT generation, TEIR parallel execution, and external framework performance.

5.1 Kernel ablation

Figure 5 summarizes the one-thread optimization progression at the two main working-set sizes. The 16 MiB case lies in the cache-assisted regime, while the 256 MiB case reaches the DRAM regime identified in Section 4.3.

Table 4. RMSNorm access-width control at 256 MiB. Each variant accesses the same 256 bytes per column.

Access form	Load instructions	GiB/s
V6: 4×64 B	4	20.94
V7x2: 2×128 B	2	24.31
V7: 1×256 B	1	24.67

The complete measurements for all intermediate variants are reported in Appendix B.1.

The first variants change arithmetic and loop structure without altering the basic one-block access pattern. Replacing the reciprocal square root in V1 has no measurable effect on throughput. Hoisting $1/N$ is similarly neutral for RMSNorm, but LayerNorm improves from 12.82 to 14.40 GiB/s at 16 MiB, corresponding to 12.3 %. At 256 MiB, the same LayerNorm change is only 0.1 %.

The first large cache-assisted gain appears with V4. Four independent accumulator chains raise RMSNorm throughput by 26.1 % and LayerNorm by 27.6 % at 16 MiB. At 256 MiB the gains fall to 6.3 % and 4.2 %, respectively. V3 unrolls the RMSNorm loop without producing a comparable improvement, while the explicit load pipelining introduced by V5 adds almost nothing beyond V4. This indicates that breaking the reduction dependency chain is the important part of the V4 design.

V6 changes the access geometry by processing four consecutive SVL row blocks together. At 256 MiB, RMSNorm increases from 10.04 to 21.05 GiB/s, a 109.7 % gain over V0. LayerNorm rises from 7.38 to 13.09 GiB/s, or 77.4 %. At 16 MiB, RMSNorm increases from 24.87 to 38.46 GiB/s and LayerNorm from 12.82 to 16.47 GiB/s. The larger relative gains at 256 MiB show that the four-block organization becomes increasingly important in the lower-bandwidth memory regime.

V7 preserves the same four-block grouping, addresses, and arithmetic as V6, but uses SME2 multi-vector accesses. At 16 MiB, V6 and V7 are effectively identical for both operators. At 256 MiB, RMSNorm rises from 21.05 to 24.63 GiB/s, a further 17.0 %, while LayerNorm increases from 13.09 to 13.52 GiB/s, or 3.3 %. Reducing the number of memory instructions is therefore most useful for RMSNorm in the larger working-set regime.

Relative to the SSVE scale-add bandwidth measured at the corresponding working-set size, RMSNorm V7 reaches 33.3 % at 16 MiB and 41.9 % at 256 MiB. LayerNorm reaches 14.2 % and 23.0 %, respectively. These percentages remain below the 1R+1W probe because the normalization kernels perform additional activation traversals and reduction arithmetic.

To examine the V6–V7 difference more closely, an additional RMSNorm control uses two 128-byte SME2 accesses between the four-load V6 organization and the single 256-byte V7 access. Table 4 reports this dedicated run.

Reducing the load count from four to two improves throughput by 16.1 %, while reducing it from two to one adds only 1.5 %. The response is therefore not proportional to the number of removed load instructions. The measurements are consistent with a threshold between four and two accesses in this loop, but they do not

Table 5. Selected LayerNorm variance-estimator errors relative to the FP64 reference. “neg.” denotes a negative FP32 variance before the square root.

Shift	Naive	Centered two-pass	Welford
0	5.09×10^{-7}	3.89×10^{-7}	2.07×10^{-7}
10^4	1.59×10^2	2.81×10^{-5}	5.41×10^{-5}
10^5	neg.	1.51×10^{-7}	8.94×10^{-5}
10^6	neg.	7.43×10^{-7}	6.71×10^{-3}

identify its microarchitectural cause.

5.2 ZA residency

The ZA variants test whether retaining activation values in SME state can reduce the cost of re-reading the input during later normalization stages. The original single-vector implementation does not improve performance: at both main working-set sizes it remains close to the early SSVE variants and well below V6 and V7.

A dedicated **MOVA** microbenchmark shows that the transfer mechanism itself can be improved substantially. The measured rate increases from 3.88 Gvec/s for single-vector transfers to 15.50 Gvec/s with the four-vector SME2 form. Rebuilding the ZA kernels with these transfers improves RMSNorm by 62 % and LayerNorm by 114 % in the dedicated experiment. The redesigned paths nevertheless remain 37 % below V7 for RMSNorm and 24 % below V7 for LayerNorm. The transfer and kernel-control results are collected in Table 10.

The remaining difference points back to the access organization established by V6. The ZA kernels process one SVL row block per column iteration, while V7 processes four adjacent blocks together. Faster ZA transfers remove much of the original transfer cost, but they do not reproduce the denser four-block access pattern. Reducing the number of ordinary input reads is therefore not sufficient on its own.

5.3 Numerical behavior

LayerNorm uses a centered two-pass variance calculation. To quantify the numerical effect of this choice, the FP32 implementation is compared with the naive $E[x^2] - \mu^2$ expression and Welford’s recurrence. The same input is shifted by increasing constants, which leaves the ideal variance unchanged while increasing cancellation in the naive expression.

The naive estimator deteriorates rapidly as the shift grows. At shifts 10^5 and 10^6 , the FP32 value of $E[x^2] - \mu^2$ becomes negative, so the subsequent square root is invalid. The centered two-pass estimator stays below 2.81×10^{-5} relative error over the sweep. Welford avoids the catastrophic cancellation of the naive expression but reaches 6.71×10^{-3} at the largest shift.

At the largest shifts, the FP32 inputs themselves become increasingly quantized. The FP64 reference is evaluated from the same FP32 values, so the comparison continues to measure estimator error on identical data even when the represented input no longer closely follows the original real-valued distribution.

The complete normalization kernels were also tested on the same stress input. For well-conditioned data, replacing exact **FSQRT**/division with **FRSQRT**E and one Newton step increases LayerNorm’s maximum output

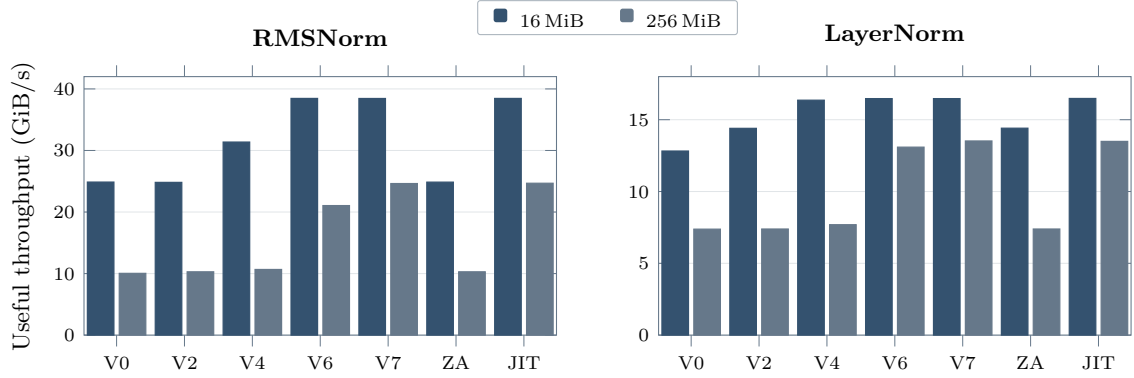


Figure 5. One-thread ablation at 16 and 256 MiB. V4 provides the largest cache-assisted gain, while four-block grouping in V6 produces the main improvement at 256 MiB. V7 retains the V6 organization and adds SME2 multi-vector accesses, which further improve RMSNorm at the larger working set.

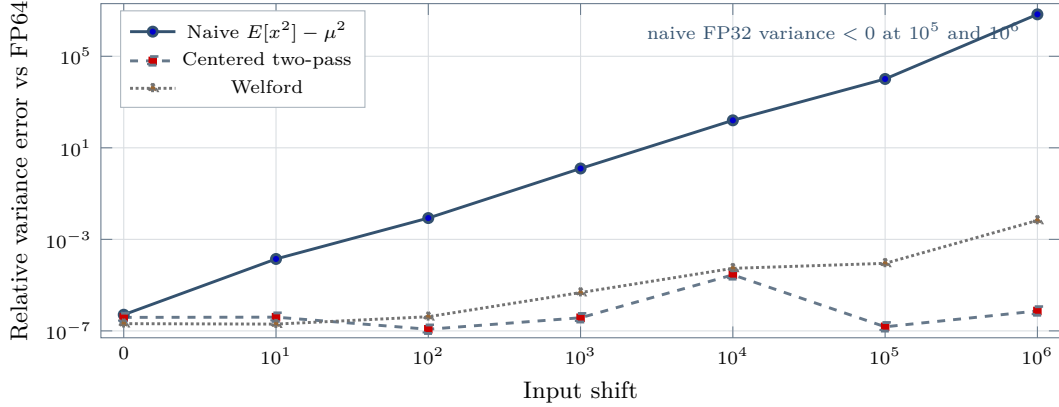


Figure 6. Relative error of three FP32 LayerNorm variance estimators against an FP64 reference as the input is shifted. The naive $E[x^2] - \mu^2$ expression eventually becomes negative because of cancellation, while the centered two-pass estimator remains stable over the tested range.

error from approximately 7.2×10^{-7} to 1.7×10^{-5} . At larger shifts, V0 and V6 converge to essentially the same error. At shift 10^5 , both are approximately 1.23×10^{-2} even though the centered variance estimate itself remains accurate. The dominant error has therefore moved to the FP32 subtraction $x - \mu$ in the output stage.

RMSNorm does not perform this subtraction and remains below 1.73×10^{-5} maximum output error over the tested sweep. Its numerical limit is different: the sum-of-squares reduction can overflow once $\sum x^2$ exceeds the FP32 range. The two operators therefore expose different numerical failure modes.

5.4 JIT generation and TEIR scaling

The JIT generates the final RMSNorm and LayerNorm kernels in approximately $4.5 \mu\text{s}$ in the measured run. The SME2 functions contain 110 and 144 emitted instruction words, respectively, and their encoding comparison with the linked hand-written implementations reports no mismatches.

The generated code also matches the throughput of the hand-written kernels. At 1024×2048 , hand-written and JIT-generated RMSNorm both reach 38.45 GiB/s in the dedicated comparison, while both LayerNorm paths reach 16.48 GiB/s. At the larger working set, the differences remain within a few tenths of a percent. Runtime generation therefore preserves the performance

of the equivalent hand-written instruction sequence. Additional samples are shown in Table 14.

The generated function pointer is retained after construction, so the emission cost is paid once and reused by later invocations. For repeated execution of the medium and large tensor shapes studied here, the steady-state kernel throughput is therefore the more relevant cost.

On the 256 MiB problem, TEIR-invoked RMSNorm reaches 24.67 GiB/s at one thread and 43.57 GiB/s at 16 threads. LayerNorm increases from 13.48 to 26.07 GiB/s. Scaling is sublinear for both operators, showing that additional workers do not translate directly into proportional throughput gains.

5.5 Framework comparison

Table 6 compares one-thread median throughput with the strongest measured PyTorch or ExecuTorch path for each operator and shape. MLC-Norm denotes the public auto-dispatched implementation, which selects V7 on the measured SME2 system. Median throughput is used because several framework configurations show substantial variation between their best and typical samples.

For LayerNorm, PyTorch eager is the strongest external path on both larger shapes. MLC-Norm reaches $2.12\times$ its median throughput at 1024×2048 and $1.87\times$ at 4096×8192 . For RMSNorm, `torch.compile` is the strongest external path, with corresponding ratios of

Table 6. One-thread median throughput in useful GiB/s. The strongest external path is selected separately for each operator and shape. Each implementation uses its native tensor layout; layout conversion is not included.

Norm	Shape	MLC-Norm	Best external	Ratio	External path
LayerNorm	1024×2048	16.42	7.76	$2.12\times$	PyTorch eager
LayerNorm	4096×8192	13.47	7.21	$1.87\times$	PyTorch eager
RMSNorm	1024×2048	38.43	6.44	$5.97\times$	<code>torch.compile</code>
RMSNorm	4096×8192	24.58	4.57	$5.38\times$	<code>torch.compile</code>

$5.97\times$ and $5.38\times$. The complete one-thread external medians are reported in Table 11.

On the measured PyTorch 2.13.0 CPU path, profiler traces show that eager LayerNorm uses PyTorch’s fused native LayerNorm operator, while eager RMSNorm is decomposed into several ATen operations. The compiled RMSNorm path is therefore substantially stronger than eager RMSNorm.

The large-shape threaded comparison is shown in Table 7.

From one to ten requested threads, MLC-Norm RMSNorm increases from 24.59 to 40.95 GiB/s, while the strongest external median increases from 4.07 to 10.68 GiB/s. LayerNorm rises from 13.53 to 24.77 GiB/s, compared with 7.23 to 10.38 GiB/s externally. At ten threads, MLC-Norm retains a $3.83\times$ advantage for RMSNorm and a $2.39\times$ advantage for LayerNorm.

Parallel execution is not beneficial at every size. At 128×64 , RMSNorm falls from 23.63 GiB/s with one thread to 1.88 GiB/s with two, and LayerNorm shows the same qualitative behavior. At this scale, the OpenMP parallel region and synchronization cost more than the kernel work they distribute. The current TEIR schedule therefore needs a size threshold before enabling row-parallel execution.

The framework measurements also show why medians are used for the headline comparison. At 1024×2048 with four threads, PyTorch eager LayerNorm records a best sample of 28.42 GiB/s but a median of 11.03 GiB/s. MLC-Norm records 24.83 GiB/s best and 24.06 GiB/s median on the same configuration. The PyTorch best sample exceeds MLC-Norm’s best sample, while its median is less than half the MLC-Norm median. A single minimum is therefore not representative of that measurement distribution.

6 Discussion

6.1 Bottlenecks change with working-set regime

The ablation shows that the limiting factor changes with working-set size. At 16 MiB, exposing independent arithmetic is particularly effective: V4 improves RMSNorm by 26.1% and LayerNorm by 27.6%. In contrast, replacing V6’s individual vector accesses with V7’s SME2 multi-vector forms produces no further gain at this size. These measurements point to reduction dependencies and arithmetic scheduling as important limits in the cache-assisted regime.

The balance changes at 256 MiB. V4 contributes only a few percent, while V6 raises RMSNorm from 10.04 to 21.05 GiB/s and LayerNorm from 7.38 to 13.09 GiB/s.

V7 then adds a further 17.0% for RMSNorm. The strongest optimization in this regime is therefore the change from one SVL row block to four adjacent blocks per feature step. It increases the amount of contiguous data accessed by each column traversal without changing the normalization algorithm itself.

The ZA result reinforces this distinction between algorithmic traffic and access organization. Retaining activations in ZA reduces ordinary input re-reads, but the resulting kernels process one SVL row block at a time. Even after replacing the original transfers with four-vector SME2 `MOVA`, they remain slower than V7. For these kernels, reducing the number of reads is less valuable than arranging those reads into the denser four-block pattern.

The optimized kernels also remain below the SSVE bandwidth measured at the corresponding working-set size. At 256 MiB, RMSNorm V7 reaches 41.9% of the scale-add reference and LayerNorm reaches 23.0%. These percentages should not be interpreted as unused bandwidth that normalization could necessarily recover. The reference performs a simple $1R+1W$ stream, whereas the normalization kernels contain additional traversals, reductions, address updates, broadcasts, and normalization arithmetic. The bandwidth curve is therefore useful for distinguishing memory regimes, but it is not a prediction of attainable normalization throughput.

The V7x2 control places a further limit on the interpretation of the SME2 result. Reducing four 64-byte loads to two 128-byte loads captures almost all of the RMSNorm improvement, while replacing those two loads with one 256-byte load adds only 1.5%. The gain is therefore not proportional to the number of removed load instructions or to the number of bytes represented by each instruction. The measurements establish a threshold-like response between four and two accesses in this loop, but do not identify its microarchitectural cause.

6.2 Granularity is a runtime decision

The four-block organization introduces a second trade-off: the access pattern that performs well on large tensors also increases the row granularity of one kernel group. On the measured M4, $4 \times$ SVL corresponds to 64 FP32 rows. When fewer than 64 rows remain, the same group-level schedule executes with inactive lanes masked by predication.

This effect is visible in the $N = 512$ crossover experiment in Table 12. V7 remains near $6.2\mu s$ from $M = 4$ through $M = 64$, while scalar execution grows with the number of rows and becomes slower between $M = 8$ and $M = 16$. The grouped kernel is therefore a poor

Table 7. Threaded median throughput at 4096×8192 in useful GiB/s. “External” is the strongest measured PyTorch or ExecuTorch median at each requested thread count.

Threads	RMSNorm			LayerNorm		
	MLC-Norm	External	Ratio	MLC-Norm	External	Ratio
1	24.59	4.07	$6.04\times$	13.53	7.23	$1.87\times$
2	29.06	9.73	$2.99\times$	18.11	10.59	$1.71\times$
4	35.71	10.15	$3.52\times$	22.72	10.74	$2.12\times$
10	40.95	10.68	$3.83\times$	24.77	10.38	$2.39\times$

choice for very small M even though it is the fastest implementation at the larger shapes.

Streaming-mode transition cost does not explain this plateau. In the control reported in Table 13, an `SMSTART/SMSTOP` round trip measures approximately 9.04 ns after subtracting the empty-loop control. Enabling ZA adds no measurable transition overhead in the same microbenchmark. A separate $M = 1$, $N = 1$ RMSNorm call has a 45.8 ns floor, while the grouped $N = 512$ measurements remain near $6.2 \mu\text{s}$. The fixed cost of entering streaming mode is therefore visible for extremely small calls, but it is far too small to account for the $N = 512$ group-level plateau.

This creates a natural opportunity for shape-dependent kernel selection. The TEIR invocation already exposes the problem dimensions to the runtime, and the JIT infrastructure can support more than one leaf implementation. A narrower row-group variant could therefore serve small M , with the four-block V7 kernel retained once enough rows are available to amortize its group granularity. The crossover should be selected from measurement rather than from a fixed architectural assumption.

The same principle applies to threading. At 128×64 , whose combined activation input and output occupy 64 KiB, RMSNorm falls from 23.63 GiB/s with one thread to 1.88 GiB/s with two. At this scale, parallel region setup and synchronization dominate the useful work. Parallel execution should therefore be enabled only once the problem is large enough to amortize those costs.

There is also a trade-off between kernel granularity and parallel decomposition. The leaf kernel prefers row chunks that align with its $4 \times \text{SVL}$ grouping, while smaller chunks can expose work to more threads. Chunk size is consequently not only a scheduling parameter: it affects how efficiently each worker can execute the selected leaf kernel. A runtime policy should consider both the available row parallelism and the preferred granularity of the generated kernel.

6.3 Scope and limitations

All architecture-specific measurements in this work were collected on one Apple M4 running macOS 15.2. The kernels derive their vector length at runtime and do not depend on a fixed 512-bit SVL, but this does not make the measured performance portable. Different SME implementations may have different vector lengths, cache hierarchies, instruction throughput, and streaming-mode costs. The optimization sequence would therefore need to be measured again on another

target.

The external comparison also reflects the layouts used by the implementations under test. PyTorch and ExecuTorch operate in their native layouts, while MLC-Norm uses the column-major organization described in Section 3.1. No layout conversion is included in the reported operator timings. The results consequently compare the available operator implementations on the measured shapes; they do not include the cost of adapting an otherwise row-major model around the MLC-Norm layout.

The PyTorch profiler result is similarly specific to the tested software configuration. In PyTorch 2.13.0, eager LayerNorm reaches the fused native LayerNorm path, while eager RMSNorm is decomposed into several ATen operations. This explains part of the large difference between their external baselines, but the same internal behavior should not be assumed for other PyTorch versions. Equivalent partitioning evidence was not obtained for ExecuTorch.

Threaded measurements depend on the operating-system scheduler as well as the requested worker count. The benchmark requests `QOS_CLASS_USER_INTERACTIVE` and records CPU time relative to wall time to observe whether workers execute concurrently, but it does not enforce or record a physical core identity for each worker. The threaded results should therefore be interpreted as measurements of this scheduler configuration, not as per-core pinned scaling measurements.

ExecuTorch 1.4.1 adds a software-specific reproducibility issue. In the tested configuration, XNNPACK lowering of RMSNorm at 1024×2048 fails intermittently. The benchmark retries the lowering step and reports the number of attempts. This behavior affects the reproducibility of that particular path and is not used to draw a broader conclusion about XNNPACK.

Finally, the project’s CI environment can compile the SME assembly but does not provide SME execution. Portable normalization, dispatch, and sanitizer tests therefore run in CI, while SME numerical tests, JIT encoding comparison, and performance measurements are executed on the M4 measurement system. A successful CI run consequently verifies the portable build and test paths, not the hardware-specific performance results.

7 Conclusion

This work studied how LayerNorm and RMSNorm should be mapped to SME and SME2 on the Apple M4, and how the resulting kernels can be incorporated into a runtime code-generation and scheduling frame-

work. The main result is that the best optimization depends strongly on the working-set regime. At 16 MiB, breaking reduction dependency chains is particularly effective, while changes to the memory-instruction form contribute little. At 256 MiB, access organization becomes dominant: processing four consecutive SVL row blocks raises RMSNorm from 10.04 to 21.05 GiB/s and LayerNorm from 7.38 to 13.09 GiB/s. SME2 multi-vector accesses then increase RMSNorm to 24.63 GiB/s, with a much smaller effect on LayerNorm.

The experiments also show that reducing nominal memory traffic is not sufficient by itself. Retaining activations in ZA removes ordinary input re-reads, but even the redesigned SME2 ZA implementation remains slower than the grouped V7 kernel. The V7x2 control similarly shows that the SME2 gain does not scale linearly with the number of removed load instructions. Together, these results point to access density, instruction structure, and reduction dependencies as interacting constraints whose importance changes with the working set.

Numerical behavior introduces a second design constraint. The centered two-pass LayerNorm variance calculation remains stable when the naive $E[x^2] - \mu^2$ formulation fails through cancellation, justifying the additional traversal despite its performance cost. At the other end of the size range, the small-shape experiments show that the four-block kernel granularity and parallel scheduling can outweigh the benefit of the optimized leaf kernel. Kernel selection must therefore account for both numerical requirements and problem size.

The JIT-generated kernels match the throughput of their hand-written counterparts while allowing the executable leaf to be selected and constructed at runtime. TEIR supplies the tensor shape, strides, and outer scheduling around that leaf, so these decisions do not have to be fixed in a single assembly entry point. On the two larger benchmark shapes, the final auto-dispatched implementation reaches 1.87–2.12× the strongest measured external LayerNorm throughput and 5.38–5.97× for RMSNorm. The next step is therefore a shape-aware policy that selects row grouping and parallelism accord-

ing to the execution regime, together with validation of the same optimization sequence on additional SME implementations.

Acknowledgment

Project-related use of generative AI is documented in the repository.

References

- [1] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer normalization,” *arXiv:1607.06450*, 2016.
- [2] B. Zhang and R. Sennrich, “Root mean square layer normalization,” in *Advances in Neural Information Processing Systems*, vol. 32, 2019.
- [3] A. Vaswani *et al.*, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, 2017.
- [4] S. Williams, A. Waterman, and D. Patterson, “Roofline: An insightful visual performance model for multicore architectures,” *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.
- [5] B. P. Welford, “Note on a method for calculating corrected sums of squares and products,” *Technometrics*, vol. 4, no. 3, pp. 419–420, 1962.
- [6] J. D. McCalpin, “Memory bandwidth and machine balance in current high performance computers,” *IEEE TCCA Newsletter*, pp. 19–25, 1995.
- [7] Apple Inc., “Scalable Matrix Extension support,” XNU Arm documentation, <https://github.com/apple-oss-distributions/xnu/blob/main/doc/arm/sme.md>.
- [8] A. Paszke *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” in *NeurIPS*, 2019.
- [9] Meta, “ExecuTorch: On-device AI inference,” <https://pytorch.org/executorch/>.

A Reproducibility

The repository and checked-in result artifacts are available at github.com/kets01/MLC-Project. The main build and benchmark sequence is:

```
cmake --preset release-submission
cmake --build build-submission -j
./build-submission/src/norm/test_norm
./build-submission/src/norm/main_norm
```

External one-thread baselines are generated with the benchmark drivers and verified with the C++ checker. A typical one-thread sequence is:

```
python bench/baselines/torch_norm_bench.py \
--outdir /tmp/mlc/torch --threads 1
python bench/baselines/executorch_norm_bench.py \
--outdir /tmp/mlc/et --threads 1
./build/bench/verify_baseline /tmp/mlc/torch
./build/bench/verify_baseline /tmp/mlc/et
```

The threaded study repeats the framework drivers at 1, 2, 4 and 10 requested threads. Each driver writes a manifest row per implementation, norm and shape. The verifier treats a missing output, a semantic mismatch or a non-finite value as a failure. This makes the set of compared configurations auditable from the artifact directory.

The internal submission artifact used for the consolidated kernel measurements records commit **98c5e79**, Release configuration, AppleClang 16.0.0, macOS 15.2, Apple M4, SME/SME2 support and SVL=16 FP32 lanes. The external one-thread artifact records clean commit **d5bfd3**. Later repository commits add the threaded comparison, documentation and build fixes; the result files remain checked in beside the scripts that produced them.

B Additional measurements

B.1 Complete kernel ablation

Table 8. Consolidated one-thread ablation in useful GiB/s. Every row passed the FP64-reference correctness gate before timing. Values are best-of- N ; medians are close for these medium and large internal-kernel cases.

Variant	RMSNorm		LayerNorm	
	16 MiB	256 MiB	16 MiB	256 MiB
Scalar reference	0.90	0.59	0.84	0.47
V0 baseline SSVE	24.87	10.04	12.82	7.38
V1 reciprocal sqrt	24.88	10.06	12.79	7.38
V2 hoisted $1/N$	24.82	10.29	14.40	7.39
V3 unroll	24.84	10.34	—	—
V4 accumulator ILP	31.37	10.67	16.36	7.69
V5 load pipelining	31.35	10.65	16.35	7.56
V6 four-block grouping	38.46	21.05	16.47	13.09
V7 SME2 multi-vector	38.45	24.63	16.47	13.52
ZA residency	24.86	10.29	14.41	7.39
JIT automatic ISA	38.46	24.68	16.48	13.49
V7 / SSVE ceiling at corresponding working-set size	33.3%	41.9%	14.2%	23.0%

B.2 Full bandwidth sweep

Table 9. Bandwidth measurements from the submission artifact. The main text uses the SSVE column; NEON is included here as auxiliary context.

Working set	NEON	SSVE
64 KiB	244.14	104.69
128 KiB	225.22	97.66
256 KiB	158.43	117.21
512 KiB	110.57	117.21
1 MiB	109.53	116.04
2 MiB	109.27	116.03
4 MiB	108.76	115.74
8 MiB	108.82	115.60
16 MiB	105.57	115.60
32 MiB	92.26	91.25
64 MiB	82.22	62.16
128 MiB	79.35	59.46
256 MiB	78.08	58.78

B.3 ZA transfer control

Table 10. Controls for the ZA redesign. Transfer rates are measured by the MOVA microbenchmark; kernel rows report relative changes from the dedicated ZA experiment.

Measurement	Result
Single-vector MOVA transfer rate	3.88 Gvec/s
Four-vector SME2 MOVA transfer rate	15.50 Gvec/s
RMSNorm redesigned ZA vs original ZA	+62%
RMSNorm redesigned ZA vs V7	−37%
LayerNorm redesigned ZA vs original ZA	+114%
LayerNorm redesigned ZA vs V7	−24%

B.4 Complete one-thread external baselines

Table 11. Median one-thread throughput in useful GiB/s for every external path reported by the checked-in baseline artifact. Each implementation uses its native tensor layout.

Norm	Shape	PyTorch eager	torch.compile	ET portable	ET XNNPACK
LayerNorm	128 × 64	12.57	3.84	4.98	4.92
LayerNorm	1024 × 2048	7.76	4.52	6.01	6.15
LayerNorm	4096 × 8192	7.21	4.40	5.83	5.89
RMSNorm	128 × 64	8.77	4.40	2.50	8.09
RMSNorm	1024 × 2048	2.90	6.44	1.89	4.36
RMSNorm	4096 × 8192	2.75	4.57	1.84	4.09

B.5 Small-shape crossover

Table 12. RMSNorm crossover against the scalar reference at $N = 512$. A V7 group covers up to 64 rows on the reference M4; inactive lanes are predicated.

M rows	Scalar (μ s)	V7 (μ s)	Speedup	Groups
1	0.250	9.291	0.03×	1
2	1.166	6.708	0.17×	1
4	2.416	6.291	0.38×	1
8	4.916	6.209	0.79×	1
16	9.916	6.250	1.59×	1
64	74.833	6.166	12.14×	1
256	422.208	25.250	16.72×	4

B.6 Runtime and JIT controls

Table 13. Streaming-transition control; the empty-loop cost is subtracted from both transition measurements.

Variant	ns/iteration	minus control
Empty loop	0.254	—
SM+ZA start/stop	9.289	9.035
SM-only start/stop	9.292	9.038

The two transition measurements are indistinguishable at this resolution, so ZA adds no measurable transition overhead in this microbenchmark.

Table 14. Hand-written and JIT-generated V7 throughput in useful GiB/s.

Norm	Shape	Hand-written	JIT
RMSNorm	128 × 64	39.58	38.56
LayerNorm	128 × 64	16.46	16.46
RMSNorm	1024 × 2048	38.45	38.45
LayerNorm	1024 × 2048	16.48	16.48
RMSNorm	4096 × 2048	27.23	27.19
LayerNorm	4096 × 2048	14.59	14.58